

```

import { useState, useEffect, useCallback } from "react";
// Google Font import via style injection
const fontLink = document.createElement("link");
fontLink.rel = "stylesheet";
fontLink.href = "https://fonts.googleapis.com/css2?family=Nunito:wght@400;500;600";
document.head.appendChild(fontLink);

// — Storage helpers —————
const KEYS = {
  expenses: "bai_expenses",
  income: "bai_income",
  tasks: "bai_tasks",
  bad_habits: "bai_bad_habits",
  good_habits: "bai_good_habits",
  ideas: "bai_ideas",
  settings: "bai_settings",
};

const load = async (key, shared = false) => {
  try {
    const r = await window.storage.get(key, shared);
    return r ? JSON.parse(r.value) : null;
  } catch {
    return null;
  }
};

const save = async (key, value, shared = false) => {
  try {
    await window.storage.set(key, JSON.stringify(value), shared);
  } catch {}
};

// — Date helpers —————
const today = () => new Date().toISOString().split("T")[0];
const thisWeekStart = () => {
  const d = new Date();
  d.setDate(d.getDate() - d.getDay() + 1);
  return d.toISOString().split("T")[0];
};
const thisMonthStart = () => {
  const d = new Date();
  return `${d.getFullYear()}-${String(d.getMonth() + 1).padStart(2, "0")}-01`;
};

```

```

const formatDate = (iso) => {
  const d = new Date(iso);
  return d.toLocaleDateString("ru-RU", { day: "numeric", month: "short" });
};

const daysSince = (iso) =>
  Math.max(1, Math.round((Date.now() - new Date(iso)) / 86400000));

// — Finance tips —————
const TIPS = [
  "Правило 50/30/20: 50% — нужды, 30% — желания, 20% — сбережения.",
  "Записывай каждую трату. Осознанность снижает импульсивные покупки на 20%.",
  "Готовь еду дома — экономия до 40% от расходов на питание.",
  "Отмени подписки, которыми не пользуешься уже 30+ дней.",
  "Покупай продукты по списку — снижает лишние траты на 25%.",
  "Кофе с собой вместо кафе: экономия ~5000₽ в месяц.",
  "Автоплатёж на накопления в день зарплаты — сначала откладывай, потом трать.",
  "Сравнивай цены в 2-3 магазинах перед крупными покупками.",
  "Доп. доход: продай вещи, которыми не пользуешься 1+ год.",
  "Фриланс в свободное время = быстрый старт дополнительного заработка.",
];

// — AI Tip via Claude —————
const getAITip = async (expenses, income, totalMonth) => {
  try {
    const res = await fetch("https://api.anthropic.com/v1/messages", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        model: "claude-sonnet-4-20250514",
        max_tokens: 1000,
        messages: [
          {
            role: "user",
            content: `Ты финансовый советник. Дай 3 конкретных, персонализированных
Данные: доход = ${income}₽/мес, расходы за месяц = ${totalMonth}₽, категории расхо
Отвечай на русском, кратко, без вступлений. Верни JSON: {"tips": ["совет1","совет
          },
        ],
      })),
    });
    const data = await res.json();
    const text = data.content?.map((c) => c.text || "").join("") || "";
    const clean = text.replace(/```json|```/g, "").trim();
    const parsed = JSON.parse(clean);
    return parsed.tips || [];
  } catch {
    return [];
  }
};

```

```

    }
};

// =====
//  MAIN APP
// =====

export default function App() {
  const [tab, setTab] = useState("finance");
  const [loaded, setLoaded] = useState(false);

  // Finance state
  const [expenses, setExpenses] = useState([]);
  const [income, setIncome] = useState(0);
  const [expInput, setExpInput] = useState({ amount: "", category: "Еда", note: "" });
  const [incomeInput, setIncomeInput] = useState("");
  const [aiTips, setAiTips] = useState([]);
  const [loadingTips, setLoadingTips] = useState(false);

  // Tasks
  const [tasks, setTasks] = useState({ day: [], week: [], month: [], year: [] });
  const [taskInput, setTaskInput] = useState({ day: "", week: "", month: "", year: "" });

  // Habits
  const [badHabits, setBadHabits] = useState([]);
  const [goodHabits, setGoodHabits] = useState([]);
  const [badInput, setBadInput] = useState("");
  const [goodInput, setGoodInput] = useState("");

  // Ideas
  const [ideas, setIdeas] = useState([]);
  const [ideaInput, setIdeaInput] = useState({ title: "", body: "" });

  // Load all data
  useEffect(() => {
    (async () => {
      const [e, inc, t, bh, gh, id] = await Promise.all([
        load(KEYS.expenses),
        load(KEYS.income),
        load(KEYS.tasks),
        load(KEYS.bad_habits),
        load(KEYS.good_habits),
        load(KEYS.ideas),
      ]);
      if (e) setExpenses(e);
      if (inc) setIncome(inc);
      if (t) setTasks(t);
      if (bh) setBadHabits(bh);
    })();
  });
}

```

```

        if (gh) setGoodHabits(gh);
        if (id) setIdeas(id);
        setLoaded(true);
    })();
}, []);

// Persist
useEffect(() => { if (loaded) save(KEYS.expenses, expenses); }, [expenses, loaded]);
useEffect(() => { if (loaded) save(KEYS.income, income); }, [income, loaded]);
useEffect(() => { if (loaded) save(KEYS.tasks, tasks); }, [tasks, loaded]);
useEffect(() => { if (loaded) save(KEYS.bad_habits, badHabits); }, [badHabits, loaded]);
useEffect(() => { if (loaded) save(KEYS.good_habits, goodHabits); }, [goodHabits, loaded]);
useEffect(() => { if (loaded) save(KEYS.ideas, ideas); }, [ideas, loaded]);

// — Finance calculations —————
const todayStr = today();
const weekStart = thisWeekStart();
const monthStart = thisMonthStart();

const sumOf = (list) => list.reduce((s, e) => s + e.amount, 0);
const expToday = sumOf(expenses.filter((e) => e.date === todayStr));
const expWeek = sumOf(expenses.filter((e) => e.date >= weekStart));
const expMonth = sumOf(expenses.filter((e) => e.date >= monthStart));

const daysInMonth = new Date().getDate();
const dailyAvg = daysInMonth > 0 ? expMonth / daysInMonth : 0;
const daysLeft = new Date(new Date().getFullYear(), new Date().getMonth() + 1, 0).getDate();
const forecastMonth = expMonth + dailyAvg * daysLeft;
const savings = income > 0 ? Math.max(0, income - forecastMonth) : 0;
const dailySavings = daysLeft > 0 ? savings / 30 : 0;

const CATS = ["Еда", "Транспорт", "Развлечения", "Одежда", "Здоровье", "Кофе/Рекреация"];

const addExpense = () => {
    if (!expInput.amount || isNaN(expInput.amount)) return;
    const newExp = {
        id: Date.now(),
        amount: parseFloat(expInput.amount),
        category: expInput.category,
        note: expInput.note,
        date: todayStr,
    };
    setExpenses((p) => [newExp, ...p]);
    setExpInput({ amount: "", category: "Еда", note: "" });
};

const fetchAiTips = async () => {

```

```

    setLoadingTips(true);
    const tips = await getAITip(expenses, income, expMonth);
    setAiTips(tips.length ? tips : [TIPS[Math.floor(Math.random() * TIPS.length)]]
    setLoadingTips(false);
};

// — Tasks —————
const addTask = (period) => {
    const text = taskInput[period].trim();
    if (!text) return;
    setTasks((p) => ({
        ...p,
        [period]: [...p[period], { id: Date.now(), text, done: false, created: toda
    }));
    setTaskInput((p) => ({ ...p, [period]: "" }));
};

const toggleTask = (period, id) => {
    setTasks((p) => ({
        ...p,
        [period]: p[period].map((t) => (t.id === id ? { ...t, done: !t.done } : t))
    }));
};

const deleteTask = (period, id) => {
    setTasks((p) => ({ ...p, [period]: p[period].filter((t) => t.id !== id) }));
};

// — Habits —————
const addBadHabit = () => {
    if (!badInput.trim()) return;
    setBadHabits((p) => [...p, { id: Date.now(), text: badInput.trim(), streak: 0
    setBadInput("");
};

const addGoodHabit = () => {
    if (!goodInput.trim()) return;
    setGoodHabits((p) => [...p, { id: Date.now(), text: goodInput.trim(), streak:
    setGoodInput("");
};

const checkHabit = (type, id) => {
    const setter = type === "bad" ? setBadHabits : setGoodHabits;
    setter((p) =>
        p.map((h) => {
            if (h.id !== id) return h;
            const yesterday = new Date();
            yesterday.setDate(yesterday.getDate() - 1);

```

```

        const yStr = yesterday.toISOString().split("T")[0];
        const newStreak = h.lastChecked === yStr ? h.streak + 1 : h.lastChecked =
        return { ...h, streak: newStreak, lastChecked: todayStr };
    })
  );
};

const deleteHabit = (type, id) => {
  if (type === "bad") setBadHabits((p) => p.filter((h) => h.id !== id));
  else setGoodHabits((p) => p.filter((h) => h.id !== id));
};

// — Ideas —————
const addIdea = () => {
  if (!ideaInput.title.trim()) return;
  setIdeas((p) => [{ id: Date.now(), ...ideaInput, date: todayStr, starred: false }, ...p]);
  setIdeaInput({ title: "", body: "" });
};

const toggleStar = (id) => {
  setIdeas((p) => p.map((i) => (i.id === id ? { ...i, starred: !i.starred } : i)));
};

const deleteIdea = (id) => setIdeas((p) => p.filter((i) => i.id !== id));

if (!loaded) return (
  <div style={{ background: "linear-gradient(160deg, #f0f7f4, #e8f4fd)", minHeight: 100px; position: relative; padding: 10px; border-radius: 10px; border: 1px solid #ccc; margin: 10px 0;}}>
    <div style={{ color: "#22c55e", fontFamily: "'Nunito', sans-serif", font-size: 14px; position: absolute; top: 10px; left: 10px; font-weight: bold;}}>
      Add New Idea
    </div>
  </div>
);

// — UI —————
const TABS = [
  { id: "finance", label: "💰 Финансы" },
  { id: "tasks", label: "✅ Задачи" },
  { id: "habits", label: "🔄 Привычки" },
  { id: "ideas", label: "💡 Идеи" },
];

return (
  <div style={S.root}>
    <div style={S.header}>
      <div style={S.headerInner}>
        <div style={S.logoArea}>
          <div style={S.logoIcon}>B</div>
          <div style={S.logoName}>BAISANGUR</div>
          <div style={S.logoSub}>Personal Dashboard</div>
        </div>
      </div>
    </div>
  </div>
);

```

```

        </div>
    </div>
    <div style={S.dateChip}>{new Date().toLocaleDateString("ru-RU", { weekd
</div>
<div style={S.tabBar}>
    {TABS.map((t) => (
        <button key={t.id} style={{ ...S.tabBtn, ...(tab === t.id ? S.tabActi
            {t.label}
        </button>
    )}}
</div>
</div>

<div style={S.content}>
    {/* === FINANCE === */}
    {tab === "finance" && (
        <div style={S.section}>
            {/* Income */}
            <div style={S.card}>
                <div style={S.cardTitle}><img alt="Russian flag" data-bbox="495 415 515 425"/> Мой доход</div>
                <div style={S.row}>
                    <input
                        style={S.input}
                        type="number"
                        placeholder="Доход в месяц (₽)"
                        value={incomeInput}
                        onChange={(e) => setIncomeInput(e.target.value)}
                    />
                    <button style={S.btn} onClick={() => { if (incomeInput) { setInco
                        Сохранить
                    </button>
                </div>
                {income > 0 && <div style={S.incomeDisplay}>{income.toLocaleString(
</div>

            {/* Stats */}
            <div style={S.statsGrid}>
                <StatCard label="Сегодня" value={expToday} accent="#ef4444" />
                <StatCard label="Неделя" value={expWeek} accent="#f97316" />
                <StatCard label="Месяц" value={expMonth} accent="#0ea5e9" />
                <StatCard label="Прогноз/мес" value={Math.round(forecastMonth)} acc
            </div>

            {income > 0 && (
                <div style={S.statsGrid}>
                    <StatCard label="Сэкономлю/мес" value={Math.round(savings)} accen
                    <StatCard label="Сэкономлю/день" value={Math.round(dailySavings)}

```

```

        <StatCard label="Ср. трата/день" value={Math.round(dailyAvg)} acc
        <StatCard label="Хватит на дней" value={income > 0 && dailyAvg >
    </div>
  )}

  {/* Add expense */}
  <div style={S.card}>
    <div style={S.cardTitle}>✚ Добавить трату</div>
    <div style={{ display: "flex", gap: 8, flexWrap: "wrap" }}>
      <input style={{ ...S.input, flex: "1 1 80px", minWidth: 80 }} type="text" value={expInput} />
      <select style={{ ...S.input, flex: "1 1 120px" }} value={expInput} >
        {CATS.map((c) => <option key={c}>{c}</option>)}
      </select>
      <input style={{ ...S.input, flex: "2 1 140px" }} placeholder="Замечание" />
      <button style={S.btn} onClick={addExpense}>Добавить</button>
    </div>
  </div>

  {/* Recent expenses */}
  {expenses.length > 0 && (
    <div style={S.card}>
      <div style={S.cardTitle}>📅 Последние траты</div>
      <div style={{ maxHeight: 260, overflowY: "auto" }}>
        {expenses.slice(0, 20).map((e) => (
          <div key={e.id} style={S.expRow}>
            <div style={S.expCat}>{e.category}</div>
            <div style={{ flex: 1, color: "#7a9e94", fontSize: 13 }}>{e.description}
            <div style={{ color: "#a3b8af", fontSize: 12 }}>{formatDate(e.date)}
            <div style={S.expAmount}>-{e.amount.toLocaleString("ru-RU")}
            <button style={S.delBtn} onClick={() => setExpenses((p) => p.filter(id => id !== e.id)}>✖
          </div>
        ))}
      </div>
    </div>
  )}

  {/* Tips */}
  <div style={S.card}>
    <div style={{ display: "flex", justifyContent: "space-between", alignItems: "center" }}>
      <div style={S.cardTitle}>🧠 Советы ИИ</div>
      <button style={{ ...S.btn, padding: "6px 14px", fontSize: 12 }} onClick={loadingTips ? "Загрузка..." : "Получить советы"}>
    </div>
  </div>
  {aiTips.length > 0 ? (
    aiTips.map((t, i) => (
      <div key={i} style={S.tipItem}>

```



```

        <span style={S.tipNum}>{i + 1}</span>
        <span style={{ color: "#2d4a3e", fontSize: 14 }}>{t}</span>
      </div>
    ))
  ) : (
    <div style={{ color: "#a3b8af", fontSize: 14, textAlign: "center" }}>
      Нажми кнопку — ИИ проанализирует твои расходы и даст персональн
    </div>
  )}
</div>
</div>
)}

{/* === TASKS === */}
{tab === "tasks" && (
  <div style={S.section}>
    {[
      { key: "day", label: "📅 На сегодня", color: "#ef4444" },
      { key: "week", label: "📅 На неделю", color: "#f97316" },
      { key: "month", label: "📅 На месяц", color: "#eab308" },
      { key: "year", label: "📅 На год", color: "#0ea5e9" },
    ].map(({ key, label, color }) => (
      <div key={key} style={S.card}>
        <div style={{ ...S.cardTitle, color }}>{label}</div>
        <div style={S.row}>
          <input
            style={S.input}
            placeholder={`Добавить задачу...`}
            value={taskInput[key]}
            onChange={(e) => setTaskInput((p) => ({ ...p, [key]: e.target.value })}
            onKeyDown={(e) => e.key === "Enter" && addTask(key)}
          />
          <button style={{ ...S.btn, background: color }} onClick={() => addTask(key)}>
            Добавить
          </button>
        </div>
        <div>
          {tasks[key].length === 0 && <div style={S.emptyHint}>Нет задач
          {tasks[key].map((t) => (
            <div key={t.id} style={S.taskRow}>
              <button
                style={{ ...S.checkBtn, ...(t.done ? S.checkDone : {}) }}
                onClick={() => toggleTask(key, t.id)}
              >
                {t.done ? "✓" : ""}
              </button>
              <span style={{ ...S.taskText, ...(t.done ? S.taskDone : {}) }}>{t.text}</span>
              <span style={S.taskDate}>{formatDate(t.created)}</span>
              <button style={S.delBtn} onClick={() => deleteTask(key, t.id)}>
                Удалить
              </button>
            </div>
          )}
        </div>
      </div>
    ))}
  </div>
)}

```

```

        </div>
      )))
    </div>
    {tasks[key].length > 0 && (
      <div style={S.progress}>
        <div style={{ ...S.progressBar, width: `${(tasks[key].filter(
          <span style={S.progressText}>{tasks[key].filter((t) => t.done
        </div>
      )}
    </div>
  )))
</div>
)}

{/* === HABITS === */}
{tab === "habits" && (
  <div style={S.section}>
    {/* Bad habits */}
    <div style={S.card}>
      <div style={{ ...S.cardTitle, color: "#ef4444" }}>🚫 Вредные привычки
      <div style={S.row}>
        <input style={S.input} placeholder="Название вредной привычки..."
        <button style={{ ...S.btn, background: "#7f1d1d" }} onClick={addB
      </div>
      {badHabits.length === 0 && <div style={S.emptyHint}>Добавь привычки
      {badHabits.map((h) => (
        <div key={h.id} style={S.habitRow}>
          <div style={S.habitInfo}>
            <div style={{ color: "#1a2e26", fontWeight: 600 }}>{h.text}</
            <div style={{ color: "#8aab9e", fontSize: 12 }}>
              {h.lastChecked === todayStr ? "✅ Сегодня устоял" : "Отметь
            </div>
          </div>
          <div style={S.streakBadge}>🔥 {h.streak}д</div>
          <button
            style={{ ...S.btn, background: h.lastChecked === todayStr ? "
            onClick={() => checkHabit("bad", h.id)}
          >
            {h.lastChecked === todayStr ? "Устоял ✓" : "Не сорвался"}
          </button>
          <button style={S.delBtn} onClick={() => deleteHabit("bad", h.id
        </div>
      )))
    </div>

    {/* Good habits */}
    <div style={S.card}>

```

```

<div style={{ ...S.cardTitle, color: "#22c55e" }}>✨ Полезные привычки
<div style={S.row}>
  <input style={S.input} placeholder="Название полезной привычки..."
  <button style={{ ...S.btn, background: "#14532d" }} onClick={addG
</div>
{goodHabits.length === 0 && <div style={S.emptyHint}>Добавь привычку
{goodHabits.map((h) => (
  <div key={h.id} style={S.habitRow}>
    <div style={S.habitInfo}>
      <div style={{ color: "#1a2e26", fontWeight: 600 }}>{h.text}</div>
      <div style={{ color: "#8aab9e", fontSize: 12 }}>
        {h.lastChecked === todayStr ? "✅ Выполнено сегодня" : "Наж
      </div>
    </div>
    <div style={{ ...S.streakBadge, background: "rgba(34,197,94,0.1"
    <button
      style={{ ...S.btn, background: h.lastChecked === todayStr ? "
      onClick={() => checkHabit("good", h.id)}
    >
      {h.lastChecked === todayStr ? "Готово ✓" : "Сделал"}
    </button>
    <button style={S.delBtn} onClick={() => deleteHabit("good", h.i
  </div>
))}
</div>
</div>
)}

{/* === IDEAS === */}
{tab === "ideas" && (
  <div style={S.section}>
    <div style={S.card}>
      <div style={S.cardTitle}>💡 Новая идея</div>
      <input style={{ ...S.input, marginBottom: 8 }} placeholder="Заголов
      <textarea
        style={{ ...S.input, minHeight: 80, resize: "vertical" }}
        placeholder="Опиши идею подробнее (опционально)..."
        value={ideaInput.body}
        onChange={(e) => setIdeaInput((p) => ({ ...p, body: e.target.valu
      />
      <button style={{ ...S.btn, marginTop: 8, width: "100%" }} onClick={
    </div>

    {ideas.length === 0 && (
      <div style={{ ...S.card, textAlign: "center", padding: 40 }}>
        <div style={{ fontSize: 48 }}>💡</div>
        <div style={{ color: "#a3b8af", marginTop: 12 }}>Здесь будут хран

```

```

    </div>
  )}

  {ideas.sort((a, b) => b.starred - a.starred).map((idea) => (
    <div key={idea.id} style={{ ...S.card, borderLeft: idea.starred ? "
      <div style={{ display: "flex", justifyContent: "space-between", a
        <div style={{ flex: 1 }}>
          <div style={{ color: "#1a2e26", fontWeight: 700, fontSize: 16
            {idea.body && <div style={{ color: "#5a7a6e", fontSize: 14, l
              <div style={{ color: "#a3b8af", fontSize: 12, marginTop: 8 }}
            </div>
          <div style={{ display: "flex", gap: 8, marginLeft: 12 }}>
            <button style={{ ...S.iconBtn, color: idea.starred ? "#22c55e
              <button style={{ ...S.iconBtn, color: "#ccd8d3" }} onClick={{
            </div>
          </div>
        </div>
      )}}
    </div>
  )}
  </div>
  )}
  </div>
);
}

```

```

// — Sub-components —————
function StatCard({ label, value, accent, unit = "P" }) {
  return (
    <div style={{ ...S.statCard, borderTop: `2px solid ${accent}` }}>
      <div style={{ color: "#7a9e94", fontSize: 11, textTransform: "uppercase", l
        <div style={{ color: accent, fontSize: 22, fontWeight: 800, marginTop: 6 }}
          {typeof value === "number" ? value.toLocaleString("ru-RU") : value}
          {typeof value === "number" && unit && <span style={{ fontSize: 12, margin
        </div>
      </div>
    );
  }
}

```

```

// — Styles —————
const S = {
  root: {
    background: "linear-gradient(160deg, #f0f7f4 0%, #e8f4fd 50%, #f5fffe 100%)",
    minHeight: "100vh",
    fontFamily: "'Nunito', sans-serif",
    color: "#1a2e26",
    maxWidth: 480,
    margin: "0 auto",
  }
}

```

```

},
header: {
  background: "linear-gradient(135deg, #1a3d2e 0%, #1e4d5a 100%)",
  borderBottom: "1px solid rgba(255,255,255,0.08)",
  position: "sticky",
  top: 0,
  zIndex: 100,
  boxShadow: "0 2px 16px rgba(30,77,90,0.18)",
},
headerInner: {
  display: "flex",
  justifyContent: "space-between",
  alignItems: "center",
  padding: "12px 16px 8px",
},
logoArea: { display: "flex", alignItems: "center", gap: 10 },
logoIcon: {
  width: 36, height: 36,
  background: "linear-gradient(135deg, #4ade80, #22c55e)",
  borderRadius: 10,
  display: "flex", alignItems: "center", justifyContent: "center",
  fontWeight: 800, fontSize: 18, color: "#fff",
  boxShadow: "0 2px 8px rgba(74,222,128,0.4)",
},
logoName: { fontWeight: 800, fontSize: 16, letterSpacing: 3, color: "#ffffff" },
logoSub: { fontSize: 10, color: "rgba(255,255,255,0.45)", letterSpacing: 2 },
dateChip: { fontSize: 11, color: "rgba(255,255,255,0.6)", background: "rgba(255,255,255,0.08)", padding: "2px 8px", borderRadius: 4 },
tabBar: { display: "flex", gap: 2, padding: "0 8px 8px", overflowX: "auto" },
tabBtn: {
  background: "transparent", border: "none", color: "rgba(255,255,255,0.5)",
  padding: "8px 12px", borderRadius: 8, cursor: "pointer",
  fontSize: 13, whiteSpace: "nowrap", transition: "all 0.2s",
  fontFamily: "'Nunito', sans-serif", fontWeight: 600,
},
tabActive: { background: "rgba(255,255,255,0.12)", color: "#ffffff", border: "1px solid #e2e2e2", borderRadius: 8 },
content: { padding: "16px 12px" },
section: { display: "flex", flexDirection: "column", gap: 12 },
card: {
  background: "#ffffff",
  border: "1px solid #e2e2e2",
  borderRadius: 16,
  padding: 16,
  boxShadow: "0 2px 12px rgba(30,77,90,0.07)",
},
cardTitle: { fontSize: 14, fontWeight: 700, color: "#1e4d5a", marginBottom: 12 },
statsGrid: { display: "grid", gridTemplateColumns: "1fr 1fr", gap: 8 },
statCard: {

```

```

    background: "#ffffff", border: "1px solid #e2e2e2",
    borderRadius: 12, padding: "12px 14px",
    boxShadow: "0 1px 6px rgba(30,77,90,0.06)",
  },
  row: { display: "flex", gap: 8 },
  input: {
    flex: 1, background: "#f4faf7", border: "1.5px solid #c8e6c9",
    borderRadius: 10, padding: "10px 12px", color: "#1a2e26",
    fontSize: 14, outline: "none",
    fontFamily: "'Nunito', sans-serif",
  },
  btn: {
    background: "linear-gradient(135deg, #22c55e, #16a34a)",
    border: "none", borderRadius: 10, padding: "10px 16px",
    color: "#fff", fontWeight: 700, cursor: "pointer",
    fontSize: 14, whiteSpace: "nowrap",
    fontFamily: "'Nunito', sans-serif",
    boxShadow: "0 2px 8px rgba(34,197,94,0.3)",
  },
  incomeDisplay: {
    marginTop: 10, color: "#16a34a",
    fontSize: 20, fontWeight: 800, textAlign: "center",
  },
  expRow: {
    display: "flex", alignItems: "center", gap: 8,
    padding: "8px 0", borderBottom: "1px solid #edf5f1",
  },
  expCat: {
    background: "#e8f5e9", color: "#15803d", fontSize: 11,
    padding: "2px 8px", borderRadius: 12, whiteSpace: "nowrap", fontWeight: 700,
  },
  expAmount: { color: "#ef4444", fontWeight: 700, fontSize: 14, whiteSpace: "nowrap",
  },
  delBtn: {
    background: "transparent", border: "none", color: "#bdc9c4",
    cursor: "pointer", fontSize: 18, lineHeight: 1, padding: "0 4px",
    flexShrink: 0,
  },
  tipItem: {
    display: "flex", gap: 10, alignItems: "flex-start",
    padding: "8px 0", borderBottom: "1px solid #edf5f1",
  },
  tipNum: {
    background: "linear-gradient(135deg, #38bdf8, #0ea5e9)",
    color: "#fff",
    width: 22, height: 22, borderRadius: "50%",
    display: "flex", alignItems: "center", justifyContent: "center",
    fontSize: 11, fontWeight: 800, flexShrink: 0,
  },

```

```

        boxShadow: "0 1px 4px rgba(14,165,233,0.3)",
    },
    taskRow: {
        display: "flex", alignItems: "center", gap: 10,
        padding: "10px 0", borderBottom: "1px solid #edf5f1",
    },
    checkBtn: {
        width: 24, height: 24, borderRadius: 7,
        border: "2px solid #cce8dc", background: "transparent",
        cursor: "pointer", display: "flex", alignItems: "center", justifyContent: "center",
        color: "transparent", fontSize: 14, flexShrink: 0, transition: "all 0.2s",
    },
    checkDone: {
        background: "#22c55e", border: "2px solid #22c55e", color: "#fff",
        boxShadow: "0 1px 6px rgba(34,197,94,0.4)",
    },
    taskText: { flex: 1, color: "#2d4a3e", fontSize: 14, fontWeight: 500 },
    taskDone: { textDecoration: "line-through", color: "#a3b8af" },
    taskDate: { color: "#a3b8af", fontSize: 11 },
    progress: { marginTop: 10, position: "relative", background: "#e8f4f0", borderR
    progressBar: { height: "100%", borderRadius: 4, transition: "width 0.4s" },
    progressText: { position: "absolute", right: 0, top: 7, color: "#a3b8af", fontS
    emptyHint: { color: "#b8cdc6", fontSize: 13, textAlign: "center", padding: "16p
    habitRow: { display: "flex", alignItems: "center", gap: 8, padding: "10px 0", b
    habitInfo: { flex: 1 },
    streakBadge: {
        background: "rgba(239,68,68,0.1)", color: "#ef4444",
        fontSize: 12, fontWeight: 800, padding: "4px 8px", borderRadius: 12,
        whiteSpace: "nowrap",
    },
    iconBtn: {
        background: "transparent", border: "none", cursor: "pointer",
        fontSize: 20, lineHeight: 1, padding: "0 4px",
    },
};

```